

Learning to Rank using XGBoost



Simon Lind · Follow

Published in Predictly on Tech · 5 min read · Mar 11, 2021

👍 302 💬 10



sci-kit learn and Pandas



Photo by [Martin Damboldt](#) from Pexels

I've gotten into horse racing. I am not a horse person, but I am an engineer. And like any engineer, I wanted to engineer a solution which could assist me in finding which horse might win a race. I decided on trying to use a technique in supervised machine learning called **Learning to Rank** to find the horses most likely to win.

During this project I tried different models for ranking. When I implemented ranking with sci-kit learns implementation of XGBoost, I found the documentation lacking and I was having a hard time progressing. After some research and a few hours of hacking, I got it working!

I thought I might spare someone some time by writing this article, a practical example of ranking using XGBoost, sci-kit learn and pandas.

What is Learning to Rank?

Before we start I would like to give a brief explanation of what Ranking is. Ranking is a subset of supervised machine learning. It differs from the more common cases classification and regression in that, instead of predicting the outcome of one data point, it takes a set of data points, a *query*, and ranks the data points.

Ranking is usually used by search engines when trying to find the most relevant result. It can also be used in product suggestion, giving relevant suggestions based on previous purchases, or in my case finding which horses are more likely to win the next race.

When ranking with XGBoost there are three objective-functions; *Pointwise*, *Pairwise*, and *Listwise*. These three objective functions are different methods of finding the rank of a set of items, and each has its own strengths and weaknesses. There are a multitude of resources explaining them in detail, but for the purpose of this article lets stick with the simple explanations.

- **Pointwise:** One instance of the set is considered at a time, use any kind of classifier or regressor to predict how relevant it is in the current query. Use each points predicted relevance to order the set.

- **Pairwise:** A pair of instances is chosen and the order of those two is predicted. Repeat this for each pair of the query to find the final order of the entire query.
- **Listwise:** Many or all instances are considered at once. Try to find the optimal order.

When trying to select the optimal objective function, cross-validation is a good method for finding the best objective function for your problem. In my case, pointwise was not at all effective. Which makes sense since a horse's chances of winning is related to its current competitors.

Data Preparation

There are a few simple things that differ from regular classification or regression, and a few pitfalls to keep in mind when preparing your data for an issue like this.

Train-Test split

First, you need a group or *query* id for your data. A column in your datasets which tells us which data points should be compared to which.

This query id must then be used when doing cross-validation or doing a train-test split. If you were to use a random shuffle split, your groups would be split and scattered between the test and training sets, making it impossible to rank.

If using a sequential split you would still run the risk of splitting one group between the test and training set.

The following example assumes you have a pandas Dataframe called **df** containing rows with feature columns, a column named **id** which identifies the group, and a target column called **rank** which contains the target.

```

1  from sklearn.model_selection import GroupShuffleSplit
2
3  gss = GroupShuffleSplit(test_size=.40, n_splits=1, random_state = 7).split(df, groups=df['id'])
4
5  X_train_inds, X_test_inds = next(gss)
6
7  train_data= df.iloc[X_train_inds]
8  X_train = train_data.loc[:, ~train_data.columns.isin(['id', 'rank'])]
9  y_train = train_data.loc[:, train_data.columns.isin(['rank'])]
10
11 test_data= df.iloc[X_test_inds]
12
13 #We need to keep the id for later predictions
14 X_test = test_data.loc[:, ~test_data.columns.isin(['rank'])]
15 y_test = test_data.loc[:, test_data.columns.isin(['rank'])]

```

[xgboost_train_test.py](#) hosted with ❤ by GitHub
[view raw](#)

When ranking, the aim is not to accurately predict the final order. Instead you are essentially trying to find what data points are relevant in the current query, and which are not. The target for Learning to Rank is a relevance score, which tells you how relevant the data point is in the current group.

In the case of horse racing the only relevant horse is the winner, the runner up can be somewhat relevant, depending on the margin to the winner. The horse which places 10:th is just as irrelevant as the one who places 11:th.

Modelling

Now that we have our training and test data, we need to pass it to the model. When performing Learning to Rank we must pass another key word argument to the model, our *group*. This arguments takes an array of the sizes of the groups in the training data.

If you have two groups in your training data, one with 10 instances and one with 7, the groups array should simply be [10, 7]. It is imperative that the training data is still sorted on the query id.

Lets add the code to get the group array for training.

```
1 from sklearn.model_selection import GroupShuffleSplit
2
3 gss = GroupShuffleSplit(test_size=.40, n_splits=1, random_state = 7).split(df, groups=df['id'])
4
5 X_train_inds, X_test_inds = next(gss)
6
7 train_data= df.iloc[X_train_inds]
8 X_train = train_data.loc[:, ~train_data.columns.isin(['id', 'rank'])]
9 y_train = train_data.loc[:, train_data.columns.isin(['rank'])]
10
11 groups = train_data.groupby('id').size().to_frame('size')['size'].to_numpy()
12
13 test_data= df.iloc[X_test_inds]
14
15 #We need to keep the id for later predictions
16 X_test = test_data.loc[:, ~test_data.columns.isin(['rank'])]
17 y_test = test_data.loc[:, test_data.columns.isin(['rank'])]
```

xbgboost_train.py hosted with ❤ by GitHub [view raw](#)

Now we are ready to fit our model, to build a ranking model, we use the *XGBRanker* module of the *xgboost* package.

```
1 import xgboost as xgb
2
3 model = xgb.XGBRanker(
4     tree_method='gpu_hist',
5     booster='gbtree',
6     objective='rank:pairwise',
7     random_state=42,
8     learning_rate=0.1,
9     colsample_bytree=0.9,
10    eta=0.05,
11    max_depth=6,
12    n_estimators=110,
13    subsample=0.75
14 )
15
16 model.fit(X_train, y_train, group=groups, verbose=True)
```

xbgboost_fit.py hosted with ❤ by GitHub [view raw](#)

Model fit using XGBRanker

Predicting

With the model trained using the training data, we are ready to make predictions on the test set. Usually we would use the models predict function on the entire test set, but the predict method does not take a group argument. So we need to do our predictions once per group.

This snippet is how I solved this issue

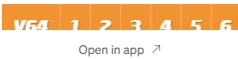
```
1 def predict(model, df):
2     return model.predict(df.loc[:, ~df.columns.isin(['id'])])
3
4 predictions = (data.groupby('id')
5               .apply(lambda x: predict(model, x)))
```

xbgboost_predict.py hosted with ❤ by GitHub [view raw](#)

And we have our prediction. I've written a small function which decorates the prediction with the horses start number. Here is a set of lists with each horses start number, in descending order according to the predicted rank. Each list corresponds to a race for V64 at Solvalla 2021-02-23. Here I am only showing the top 50%.

```
1 [1, 6, 9, 8, 4]
2 [3, 8, 2, 4, 11, 10]
3 [2, 6, 8, 5]
4 [10, 11, 8, 6]
5 [6, 10, 4, 2]
6 [3, 4, 5, 11]
```

And the actual results:



```
1 [1*, 6, 9, 8, 4]
2 [3*, 8, 2, 4, 11, 10]
3 [2, 6*, 8, 5]
4 [10, 11, 8, 6*]
5 [6, 10*, 4, 2]
6 [3*, 4, 5, 11]
```

The model is able to pick the top horse in 3/6 races and top two in 5/6!

I hope this short article will save you some time in your ranking project, whether it be search results, product suggestions or horse betting!

- Xgboost
- Data Science
- Machine Learning
- Scikit Learn
- Pandas



Published in Predictly on Tech

69 Followers · Last published Apr 3, 2024

Discoveries made while delivering technical solutions

Follow



Written by Simon Lind

86 Followers · 5 Following

Master of Science in Biotechnology Engineering with focus Bioinformatics. Cloud + ML + Data + Python + Java.

Follow

Responses (10)



What are your thoughts?

Respond



suresh kumar Gorakala
over 3 years ago

...

Hi Simon, nice work. Can you please put a snapshot of the data, so that we can see how the data looks like?

👍 95

Reply



masoud parpanchi
about 2 years ago

...

where is the code and data?

👍 53

Reply



leobol
over 3 years ago

...

Hi,

What does the rank column represent? The horses' ids ?

Could you explain how you mapped the floats predicted values to the final result ?

👍 4 💬 1 reply

Reply

See all responses

More from Simon Lind and Predictly on Tech



In Predictly on Tech by Simon Lind

Serve any XGBoost model with FastAPI in less than 40 lines

Speed up model testing by easily serving them using FastAPI

Apr 2, 2022 15 2



In Predictly on Tech by Patrik Hörlin

Spring Data JPA — batching using Streams

Spring Data JPA is great for rapid development but special care must be taken...

Sep 29, 2021 163 6



In Predictly on Tech by Patrik Hörlin

Complex JPA queries with Specifications

Perform complex and dynamic queries of JPA entities that span child collections using...

May 15, 2022 86 1



In Predictly on Tech by Simon Lind

Getting started with Machine Learning on GCP — Part 2: Making...

Now that we have our raw data in a BigQuery table we can begin exploring and getting to...

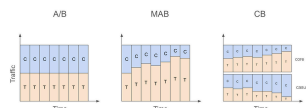
Dec 16, 2020 58



See all from Simon Lind

See all from Predictly on Tech

Recommended from Medium



In Towards Data Science by Ugur Yildirim

An Overview of Contextual Bandits

A dynamic approach to treatment personalization

Feb 2 336 1



Jessica Stillman

Jeff Bezos Says the 1-Hour Rule Makes Him Smarter. New...

Jeff Bezos's morning routine has long included the one-hour rule. New...

Oct 30 15.7K 394



Lists



Predictive Modeling w/ Python

20 stories · 1718 saves



Natural Language Processing

1856 stories · 1484 saves



Practical Guides to Machine Learning

10 stories · 2090 saves



data science and AI

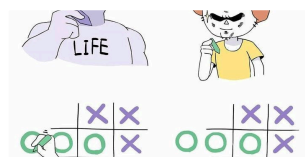
40 stories · 299 saves



In DataDrivenInvestor by Austin Starks

I used OpenAI's o1 model to develop a trading strategy. It is...

It literally took one try. I was shocked.



In Biased-Algorithms by Amit Yadav

SHAP Values vs Feature Importance

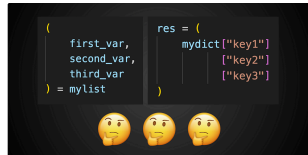
If you think you need to spend \$2,000 on a 120-day program to become a data scientist...



📺 In Modern AI by Diego Ruiz

Course: Become a Quant Trader with Python—Chapter 1: How to...

Finding Your Edge with Professional Data Sources



📺 In Level Up Coding by Liu Zuo Lin

12 Production-Grade Python Code Styles I've Picked Up From Work

Read Free...

See more recommendations